AIA3: Large Language Models (LLMs) Deep Dive

Module Introduction and Theoretical Foundations

Large Language Models represent the culmination of scaling laws, where compute, data, and architecture converge to produce systems exhibiting **emergent abilities**—capabilities not explicitly trained but arising from scale (Wei et al., 2022). This module dissects LLMs from tokenization to inference optimization, providing the mathematical and engineering foundation for building/deploying production systems.

Key Theoretical Pillars:

1. **Universal Computation**: Transformers as general function approximators
2. **Scaling Laws**: Predictable loss reduction with compute
3. **Emergence**: Phase transitions in capability curves
4. **Information Theory**: Next-token prediction as compression

Model Family Tree:

text

GPT-1 (117M) → GPT-3 (175B) → GPT-4 (1.8T?)

↓ PaLM → PaLM-2 → Gemini

LLaMA → LLaMA-2 → Mixtral (MoE)

Compute Budgets:

text

GPT-3: 3.14×10^23 FLOPs

LLaMA 65B: 1.4×10^24 FLOPs

Frontier models: 10^25+ FLOPs

(Figure 1: Compute scaling vs. performance on BIG-Bench)

Section 1: Tokenization - Subword Theory and Implementation

Why Subword? Open vocabulary problem: 1M+ English words, OOV handling.

Byte Pair Encoding (BPE) Algorithm:

1. **Initialization**: Corpus → character tokens
2. **Pair frequency counting**
3. **Merge most frequent pair → new token**
4. **Repeat until vocab size V**

Mathematical Formulation:

text

P(pair) = count(pair) / total_pairs

Greedy merge: argmax_P(pair_t)

SentencePiece Implementation (Kudo 2018):

- Treats whitespace as token ✓
- Language-agnostic ✓
- BPE + Unigram ✓

Production Tokenizer Code:

python

```python
import tiktoken
from typing import List


class LLMTok:
    def __init__(self, model_name: str = "gpt2"):
        self.enc = tiktoken.get_encoding(model_name)
        self.vocab_size = self.enc.n_vocab

    def encode(self, text: str) -> List[int]:
        """Text → token IDs"""
        return self.enc.encode(text)

    def decode(self, tokens: List[int]) -> str:
        """Token IDs → text"""
        return self.enc.decode(tokens)

    def n_bytes(self, text: str) -> int:
        """Compression ratio diagnostic"""
        tokens = self.encode(text)
        return len(text.encode('utf-8')) / len(tokens)

# Usage & Analysis
```

```
tok = LLMTok("cl100k_base")  # GPT-4 tokenizer
text = "antidisestablishmentarianism"
print(f"Tokens: {tok.encode(text)}")
print(f"Compression: {tok.n_bytes(text):.1f} bytes/token")
```

Vocab Size Scaling:

text

GPT-2: 50k → GPT-4: 100k → Gemini: 250k+

Tradeoff: Coverage vs sequence length

Multilingual Tokenization Theory:

text

English: ~2 chars/token

Mandarin: ~1.5 chars/token

Code: ~4 chars/token

Section 2: Embeddings - Positional and Semantic Encoding

Token Embeddings Matrix:

text

E ∈ ℝ^(V × d_model), V=vocab_size, d_model=4096-16384

Initialized: xavier_uniform ~ N(0, 1/√d_in)

Positional Encoding Evolution:

1. **Sinusoidal (Vaswani 2017):**

text

PE(pos, 2i) = sin(pos / 10000^(2i/d))

PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

Theory: Bounded [-1,1], periodic, relative position encoding

1. **Learned Absolutes** (GPT): Overfits training lengths
2. **RoPE (Rotary, Su 2021) - State of Art:**

text

Relative distance preservation via rotation matrices

q_m^(i) ← R_m^(i) q_m^(i)

Key Property: q_m · q_n = q'_m · q'_n preserves angles

RoPE Implementation:

python

```python
import torch

def apply_rope(q, k, max_seq_len=2048, dim=128):
    """Rotary Position Embeddings"""
    inv_freq = 1.0 / (10000 ** (torch.arange(0, dim, 2).float() / dim))
    t = torch.arange(max_seq_len).float().unsqueeze(1) * inv_freq.unsqueeze(0)

    freqs = torch.cat((t.cos(), t.sin()), dim=-1)

    # Apply rotation
    q_rot = einops.rearrange(q, '... (h d) -> ... h d', d=dim//2)
    q_rot = (q_rot * freqs[:q_rot.shape[-2]]).flatten(-2)

    k_rot = einops.rearrange(k, '... (h d) -> ... h d', d=dim//2)
    k_rot = (k_rot * freqs[:k_rot.shape[-2]]).flatten(-2)

    return q_rot, k_rot
```

Embedding Normalization:

text

Pre-LN: LayerNorm(Embedding(x))

Post-LN: LayerNorm(Attention(LayerNorm(x)))

RMSNorm (preferred): x / RMS(x) * g (no affine shift)

Section 3: Transformer Architecture - Layer by Layer

Single Decoder Layer (Llama-style):

text

1. RMSNorm1(x)
2. Self-Attention → residual add
3. RMSNorm2(x)

4. SwiGLU FFN → residual add

Causal Self-Attention with GQA:

text

Multi-Query Attention (MQA): 1 KV head

Grouped-Query (GQA): G KV heads for H query heads

Complete Attention Block:

python

```python
class CausalSelfAttention(nn.Module):
    def __init__(self, dim, n_heads=8, max_context=2048):
        super().__init__()
        self.scale = dim ** -0.5
        self.heads = n_heads
        head_dim = dim // n_heads

        self.to_qkv = nn.Linear(dim, dim * 3, bias=False)
        self.to_out = nn.Linear(dim, dim)
        self.rope = RoPE(dim=head_dim, max_seq_len=max_context)

    def forward(self, x):
        b, t, c = x.shape
        qkv = self.to_qkv(x).chunk(3, dim=-1)
        q, k, v = map(lambda t: rearrange(t, 'b t (h d) -> b h t d', h=self.heads), qkv)

        # RoPE
        q, k = self.rope(q, k)

        # Causal attention
        dots = torch.matmul(q, k.transpose(-1, -2)) * self.scale

        # Causal mask
        mask = torch.full((t, t), float('-inf'), device=x.device)
```

```
    mask = torch.triu(mask, diagonal=1)
    dots.masked_fill_(mask == float('-inf'), float('-inf'))

    attn = dots.softmax(dim=-1)
    out = torch.matmul(attn, v)

    out = rearrange(out, 'b h t d -> b t (h d)')
    return self.to_out(out)
```

SwiGLU Feedforward (Shazeer 2020):

text

$FFN(x) = (x W_1 \odot Swish(x W_3)) W_2$

Better than ReLU/GELU empirically

Layer Scaling:

text

$Params \approx 12 L d_model^2$ (L=layers)

LLaMA-70B: L=80, d_model=8192

Section 4: Pretraining - Next Token Prediction Theory

Causal Language Modeling:

text

$L(\theta) = -\Sigma_{t=1}^T \log p_\theta(x_t \mid x_{<t})$

$= -\Sigma_{t=1}^T x_t \cdot \log(softmax(W_{LM} h_L^t))$

Scaling Laws (Kaplan 2020, Hoffmann 2022):

text

$L(N) \approx (N_c / N)^\alpha$

Optimal: tokens $D \approx 20 \times$ params N

Data Mixture Optimization:

text

Code: 10-20% (sharpens reasoning)

Math: 5% (precision)

Dialogue: 10% (helpfulness)

Optimizer Recipe (LLaMA):

text

AdamW(90/0.95/1e-8), LR=3e-4 peak

Cosine decay + linear warmup 2%

bfloat16, gradient checkpointing

Production Training Loop:

python

```python
def train_step(model, batch, optimizer):
    tokens = batch['input_ids']  # (B, T)
    logits = model(tokens)[:, :-1]  # Predict next tokens
    targets = tokens[:, 1:]       # Shifted

    loss = F.cross_entropy(
        logits.reshape(-1, vocab_size),
        targets.reshape(-1),
        ignore_index=-100
    )

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()

    return loss.item()
```

Section 5: Fine-Tuning Pipeline - SFT to Alignment

Instruction Tuning Format:

text

<s>[INST] <<SYS>> system prompt <</SYS>> user prompt [/INST] assistant response </s>

Parameter Efficient Fine-Tuning:

LoRA (Hu 2021):

text

$W = W_o + \Delta W = W_o + B A$

$B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times k)}$, $r=8\text{-}64 \ll \min(d,k)$

QLoRA (Dettmers 2023): 4-bit + double quantization

Alignment Methods:

1. **RLHF Pipeline:**

text

reward_model ← SFT on preferences

policy ← PPO(r_model, KL_penalty)

1. **DPO (Rafailov 2023) - Direct Preference:**

text

L_DPO = -log sigmoid(β log(π(y_w|x)/π_ref(y_w|x) - log(π(y_l|x)/π_ref(y_l|x))))

No RL required!

Section 6: Inference Optimization and Deployment

KV Cache Mechanics:

text

Memory: $O(B L^2 d_{head}) \rightarrow O(B L d_{head})$ with cache

PagedAttention: VMAs for cache eviction

Quantization Techniques:

text

GPTQ: Post-train 4-bit (Frantar 2022)

AWQ: Activation-aware (Lin 2023)

NF4: NormalFloat4 distribution

Continuous Batching (vLLM):

text

Dynamic batching during generation

10-50x throughput vs naive

Speculative Decoding:

text

Draft model → verify with target

2-3x speedup

Complete Inference Engine:

python

```python
class LLMEngine:
    def __init__(self, model_path):
        self.model = AutoModelForCausalLM.from_pretrained(
            model_path, torch_dtype=torch.bfloat16, device_map="auto"
        )
        self.tokenizer = AutoTokenizer.from_pretrained(model_path)

    @torch.no_grad()
    def generate(self, prompt, max_new=512, temp=0.8):
        inputs = self.tokenizer(prompt, return_tensors="pt")
        input_ids = inputs.input_ids.cuda()

        for _ in range(max_new):
            logits = self.model(input_ids)[:, -1, :]
            if temp > 0:
                logits = logits / temp
                probs = F.softmax(logits, dim=-1)
                next_token = torch.multinomial(probs, num_samples=1)
            else:
                next_token = logits.argmax(-1, keepdim=True)

            input_ids = torch.cat([input_ids, next_token], dim=-1)

        return self.tokenizer.decode(input_ids[0])
```

Section 7: Evaluation, Diagnostics, and Research Frontiers

Comprehensive Benchmarks:

text

MMLU (57 subjects): 60%+ = expert level

HumanEval: Code generation

GSM8K: Grade-school math

Diagnostic Tools:

text

Perplexity: exp(average NLL)

Calibration: ECE = Σ |acc_bin - conf_bin|

Research Frontiers:

1. **Test-Time Compute**: o1-style reasoning chains
2. **Synthetic Data**: Self-improvement loops
3. **Mixture of Agents**: Ensemble experts

Career Project: Fine-tune LLaMA-7B on custom domain → deploy with vLLM